Einführung in die Programmiersprache C

Embedded Systems Engineering – Vorlesung

Prof. Dr.-Ing. Sebastian Schlesinger

Fachbereich 2 – Duales Studium Wirtschaft & Technik
HWR Berlin

Wintersemester 2026

Gliederung

1 Geschichte und Entwicklung von C

2 Grundkonstrukte

3 Zeiger (Pointer)

4 Call by Value und Speicherverwaltung

5 Objektorientierung in C simulieren

6 Zusammenfassung

Geschichte und Entwicklung von C

Ursprünge: Von BCPL zu C

Die Entwicklungslinie:

- 1967 – Martin Richards: BCPL (*Basic Combined Programming Language*), typlos, für Systemprogrammierung
- 1969 – Ken Thompson: B bei Bell Labs, vereinfachtes BCPL für die PDP-7
- 1972 – Dennis Ritchie: C – direkt gekoppelt an die Reimplementierung von Unix auf der PDP-11
- 1978 – *The C Programming Language*, Kernighan & Ritchie („K&R“) – informeller Standard

Schlüsselpersonen

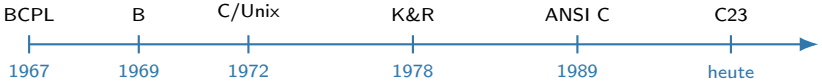
Dennis M. Ritchie (1941–2011)
Schöpfer von C, Miterfinder von Unix

Ken Thompson (*1943)
Miterfinder von Unix und B

Brian W. Kernighan (*1942)
Co-Autor von K&R C

Zitat Ritchie

„C is quirky, flawed, and an enormous success."



Die C-Standards im Überblick

Standard	Jahr	Wichtigste Neuerungen
K&R C	1978	Ursprüngliche informelle Definition
ANSI C / C89	1989	Erster ISO-Standard (ISO/IEC 9899:1990)
C95	1995	Trigraphs, `wchar.h`, Erweiterungen
C99	1999	`//`-Kommentare, `bool`, VLAs, `stdint.h`, gemischte Deklarationen, `inline`
C11	2011	`_Atomic`, `<threads.h>`, `_Generic`, anonyme Structs
C17 / C18	2018	Bugfixes, keine neuen Features
C23	2023	`nullptr`, `bool` als Keyword, `#embed`, `typeof`, Attribute

Relevanz für Embedded Systems:

- Toolchains (GCC ARM, IAR, Keil) unterstützen primär C99 und C11
- MISRA-C (2012, 2023) schreibt ein sicheres Subset vor

Kompilierung mit Standard

```
gcc -std=c11 -Wall -Wextra -o prog prog.c
```

Immer den Standard explizit angeben – der Compiler-Default kann sich zwischen Versionen ändern!

Warum C für Embedded Systems?

Stärken:

- Hardwarenähe: direkte Speicher- und Registerzugriffe über Pointer
- Effizienz: minimaler Laufzeit-Overhead, kein Garbage Collector
- Deterministismus: kein JIT, vorhersagbares Timing – kritisch für Echtzeitsysteme
- Portabilität: läuft auf nahezu jeder Prozessorarchitektur (ARM, RISC-V, AVR, MSP430 . . .)
- Reife Toolchain: GCC, Clang, LLVM, IAR, MISRA-Checker, statische Analysetools

Herausforderungen:

- Kein automatisches Speichermanagement
- Fehleranfällig: Buffer Overflows, Dangling Pointers, undefined Behavior
- Kein eingebautes OOP, kein Exception Handling
- Kein nativer Modulmechanismus (nur `#include`)

Leitsatz

C gibt dem Programmierer *alle* Freiheiten – und damit *alle* Verantwortung.

Grundkonstrukte

Struktur eines C-Programms

```c
/* Präprozessor-Direktiven */
#include <stdio.h>      /* I/O Standardbibliothek */
#include <stdint.h>     /* Exakte Integer-Typen */
#define MAX_SIZE  128   /* Makro (keine Variable!) */
/* Globale Variable (Filescope) */
static int counter = 0;
/* Funktionsprototyp (Deklaration) */
void print_hello(const char *msg);

/* Hauptprogramm */
int main(void) {
    print_hello("Hello, ESE!");
    return 0;   /* 0 = Erfolg, != 0 = Fehler */
}
/* Funktionsdefinition */
void print_hello(const char *msg) {
    printf("%s\n", msg);
}
```

Übersetzungsprozess:

1. Präprozessor (`cpp`): `#include`, `#define`, bedingte Kompilierung
2. Compiler (`cc1`): C → Assembler (`.s`)
3. Assembler (`as`): Assembler → Objektcode (`.o`)
4. Linker (`ld`): `.o` + Libs → Executable

Wichtige Compiler-Flags

`-Wall -Wextra` – alle Warnungen
`-O2` – Optimierung
`-g` – Debug-Info
`-std=c11` – C-Standard

Ganzzahl-Typen (plattformabhängig!)

`char`	1 Byte	−128 . . . 127
`short`	2 Byte	±32 767
`int`	2–4 B*	
`long`	4–8 B*	
`long long`	8 Byte	$\pm 9{,}2 \cdot 10^{18}$

*Größe plattformabhängig!

Portable Typen – `<stdint.h>` (C99)

`int8_t`, `uint8_t` – 8 Bit
`int16_t`, `uint16_t` – 16 Bit
`int32_t`, `uint32_t` – 32 Bit
`int64_t`, `uint64_t` – 64 Bit

Gleitkomma- und sonstige Typen

`float` – 4 Byte, ≈7 Dezimalstellen
`double` – 8 Byte, ≈15 Stellen
`void` – kein Typ
`bool` / `_Bool` – wahr/falsch (C99)

```
#include <stdint.h>
#include <stdbool.h>

uint8_t   reg   = 0xFF;     /* 8-Bit Register */
int32_t   count = -1;
float     volt  = 3.3f;     /* f: float -Literal */
double    pi    = 3.14159265358979;
char      c     = 'A';      /* = 65 = 0x41 */
bool      flag  = true;
```

Operatoren

Arithmetisch:

```c
int a = 10, b = 3;
a + b    /* 13 */
a - b    /* 7  */
a * b    /* 30 */
a / b    /* 3 (int!) */
a % b    /* 1 (Modulo)*/
```

Vergleich / Logik:

```c
a == b   /* falsch: 0 */
a != b   /* wahr: 1  */
a > b    /* 1        */
a && b   /* logisch  */
a || b
!a
```

Bitweise (für Embedded!):

```c
uint8_t r = 0b10110100;

r & 0x0F    /* AND:  0x04 */
r | 0x01    /* OR:   0xB5 */
r ^ 0xFF    /* XOR:  0x4B */
~r          /* NOT:  0x4B */
r << 2      /* Links shift */
r >> 1      /* Rechts shift*/
/* Bit setzen */
r |= (1u << 3);
/* Bit löschen */
r &= ~(1u << 3);
/* Bit testen */
if (r & (1u << 3)) { }
```

Zuweisung und Sonstiges:

```c
a += 5;  a -= 5;
a *= 2;  a /= 2;
a &= 0xF0;
a |= 0x01;
a++;    /* Postfix */
++a;    /* Präfix  */
a--;
/* Ternär: */
int m = (a>b)?a:b;

/* sizeof */
sizeof(int32_t) /* 4 */
sizeof(arr)/
  sizeof(arr[0])/* Anz. Elemente */
```

Kontrollstrukturen

Bedingte Ausführung:

```c
if (voltage > 3.0f) {
    set_led(GREEN);
} else if (voltage > 2.0f) {
    set_led(YELLOW);
} else {
    set_led(RED);
    shutdown();
}
/* Switch für Zustandsmaschinen */
switch (state) {
  case STATE_IDLE:
    handle_idle();    break;
  case STATE_RUN:
    handle_run();     break;
  case STATE_ERROR:
    handle_error();   break;
  default:
    /* sollte nie eintreten */
    assert(false);    break;
}
```

Schleifen:

```c
/* For-Schleife (Zählschleife) */
for (uint32_t i = 0; i < N; i++) {
    buffer[i] = 0u;
}
/* While (kopfgesteuert) */
while (!uart_tx_ready()) {
    /* warte auf Hardware */
}
/* Do-While (mindestens einmal) */
uint16_t sample;
do {
    sample = adc_read();
} while (sample < THRESHOLD);
/* break / continue */
for (int i = 0; i < 100; i++) {
    if (i == 50) break;    /* Abbruch */
    if (i % 2)  continue; /* Sprung  */
    process(i);
}
```

Funktionen und Speicherklassen

```c
/* Deklaration (Header) */
int32_t clamp(int32_t val,
              int32_t lo, int32_t hi);
/* Definition */
int32_t clamp(int32_t val,
              int32_t lo, int32_t hi) {
    if (val < lo) return lo;
    if (val > hi) return hi;
    return val;
}
/* Rekursion */
uint64_t factorial(uint32_t n) {
    return (n <= 1u) ? 1u
                     : n * factorial(n-1u);
}
/* Statische lokale Variable */
uint32_t next_id(void) {
    static uint32_t id = 0u;
    return ++id;    /* persistiert! */
}
```

Speicherklassen und Qualifier:

- `auto` – lokal, Stack (Standard)
- `static` – persistiert / Filescope
- `extern` – externe Bindung
- `inline` – Compiler-Inlining-Hint
- `volatile` – externe Manipulation möglich; keine Compiler-Optimierung

`volatile` – wichtig in Embedded!

```c
/* Ohne volatile: Compiler könnte
   Schleife wegoptimieren!        */
volatile uint32_t *timer_reg =
    (volatile uint32_t*)0x40001000;

while (*timer_reg < 1000u) {
    ; /* warte */
}
```

Arrays, Strings und zusammengesetzte Typen

Arrays:

```c
uint8_t buf[8]    = {0};        /* 0-initialisiert */
int     fib[]     = {1,1,2,3,5,8}; /* Größe auto */
int     mat[3][3]= {{1,0,0},
                    {0,1,0},
                    {0,0,1}};
/* Größe zur Laufzeit (C99 VLA) */
void fill(size_t n, int arr[n]);
```

Strings (Null-terminierte char-Arrays):

```c
char name[] = "ESE";  /* {'E','S','E','\0'} */
char buf[32];

/* Sichere Funktionen (string.h) */
strncpy(buf, name, sizeof(buf)-1);
buf[sizeof(buf)-1] = '\0';  /* sicher! */
size_t len = strlen(name);  /* 3 */
```

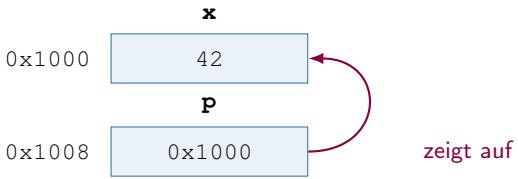
Struct und Enum:

```c
typedef enum {
    STATE_IDLE = 0,
    STATE_RUNNING,
    STATE_ERROR
} State_t;

typedef struct {
    uint32_t  id;
    float     temperature;
    State_t   state;
} Sensor_t;

/* Designated Initializer (C99) */
Sensor_t s = {
    .id          = 1u,
    .temperature = 23.5f,
    .state       = STATE_IDLE
};

printf("T=%.1f\n", s.temperature);
s.state = STATE_RUNNING;
```

Zeiger (Pointer)

Grundlagen: Was ist ein Pointer?

```c
int  x = 42;
int *p = &x;    /* p enthält Adresse von x */
printf("Wert x:     %d\n",  x);    /* 42 */
printf("Adresse x: %p\n", &x);    /* 0x.. */
printf("Pointer p: %p\n",  p);    /* = &x */
printf("*p:         %d\n", *p);    /* 42 */
*p = 100;  /* schreibt über Pointer */
printf("x = %d\n", x);     /* 100  */
/* Pointer-Typen */
char     *pc;    /* Zeiger auf char     */
uint8_t *pu;    /* Zeiger auf uint8_t  */
Sensor_t *ps;   /* Zeiger auf Struct   */
/* Struct-Member über Pointer */
Sensor_t  sensor = {.id=1};
Sensor_t *sp = &sensor;
sp->id = 2;            /* = (*sp).id = 2 */
```



Operatoren

`&var` – Adresse von `var`
`*ptr` – Wert an Adresse
`ptr->feld` – Strukturfeld via Ptr
`(*ptr).feld` – äquivalent

Goldene Regel

Pointer *immer* initialisieren!
Uninitialisiert = Undefined Behavior.

Pointer-Arithmetik und Arrays

```c
int arr[5] = {10, 20, 30, 40, 50};
int *p = arr;          /* = &arr[0] */

printf("%d\n", *p);        /* 10  */
printf("%d\n", *(p+1));    /* 20  */
printf("%d\n", *(p+4));    /* 50  */

p++;  /* p zeigt jetzt auf arr[1] */
printf("%d\n", *p);        /* 20  */
/* Äquivalente Schreibweisen */
arr[2]   ==  *(arr + 2)   /* true */
&arr[2]  ==  arr + 2      /* true */
/* Pointer-Differenz */
int *q   = &arr[4];
ptrdiff_t d = q - p;    /* = 3   */
/* Pointer auf Pointer */
int   x  = 5;
int  *p2 = &x;
int **pp = &p2;
**pp = 99;              /* x = 99 */
```

Skalierung der Arithmetik:

`p + n` springt um
$n \times$ `sizeof(*p)` Bytes

Array vs. Pointer – wichtiger Unterschied:

```c
int arr[5];
int *p = arr;

sizeof(arr) /* 20 Byte (5*4) */
sizeof(p)   /*  8 Byte (Ptr) */
```

Häufige Anwendung:

```c
/* Iteriere mit Pointer */
uint8_t buf[8];
uint8_t *end = buf + sizeof(buf);
for (uint8_t *it = buf; it < end;
     it++)
     *it = 0xFF;
```

const, void* und NULL-Pointer

const-Qualifikation:

```c
int x = 5;

/* Pointer auf const-Wert (read-only) */
const int *p1 = &x;
/* *p1 = 10; */     /* Fehler! */
p1 = NULL;          /* OK      */
/* Const Pointer (Adresse unveränderlich) */
int *const p2 = &x;
/* p2 = NULL; */    /* Fehler!  */
*p2 = 10;           /* OK       */
/* Beides const */
const int *const p3 = &x;
```

void* – generischer Pointer:

```c
void  *generic;
int    a = 42;
generic = &a;  /* kein Cast nötig */
int b = *(int*)generic; /* Cast beim Lesen */
```

NULL-Pointer:

```c
#include <stddef.h>
int *p = NULL;  /* Zeigt auf "nichts" */

if (p != NULL) {
    /* nur dann dereferenzieren! */
    printf("%d\n", *p);
}
```

Sicherheitsregeln für Pointer

1. Pointer vor Nutzung auf `NULL` prüfen
2. Nach `free()`: `ptr = NULL` setzen
3. Kein Pointer auf lokale (Stack-)Variable zurückgeben!
4. Array-Grenzen *immer* prüfen
5. `const` so oft wie möglich verwenden

Funktionspointer

```c
/* Zwei Funktionen gleicher Signatur */
int add(int a, int b) { return a + b; }
int mul(int a, int b) { return a * b; }

/* Funktionspointer-Typ per typedef */
typedef int (*BinOp_t)(int, int);

/* Direktverwendung */
BinOp_t op = add;
printf("%d\n", op(3, 4));    /* 7  */
op = mul;
printf("%d\n", op(3, 4));    /* 12 */

/* Als Parameter (Callback-Muster) */
int apply(BinOp_t f, int x, int y) {
    return f(x, y);
}
printf("%d\n", apply(add, 10, 5)); /* 15 */

/* Dispatch-Tabelle */
BinOp_t ops[] = {add, mul};
printf("%d\n", ops[1](3, 4));      /* 12 */
```

Syntax – Merkhilfe:

`int (*fp)(int, int)`

Lies: `fp` ist ein Pointer auf eine Funktion, die zwei `int` erhält und `int` zurückgibt.

Typischer Einsatz in Embedded:

- ISR-Registrierung und HAL-Callbacks
- Zustandsmaschinen als Dispatch-Tabelle
- Treiber-Interfaces (open/read/write/ioctl)
- Plugin- und Middleware-Architekturen

Immer `typedef`!

Typumwandlung (Casting)

Implizite Konvertierungen:

```
int    i = 42;
float  f = i;        /* int->float: OK */
double d = f;        /* float->double: OK */
/* GEFÄHRLICH -- stille Kürzung! */
int    big   = 300;
uint8_t small = big;  /* 300%256 = 44! */
/* Integer Promotion: klein -> int */
uint8_t a = 200u, b = 100u;
uint8_t c = a + b; /* 300 -> 44 ! */
```

Expliziter Cast:

```
float x = 3.9f;
int   n = (int)x;  /* 3 (trunc!) */
/* Hardware-Registerzugriff */
volatile uint32_t *GPIOA =
    (volatile uint32_t *)0x40010800UL;
*GPIOA |= (1u << 5);  /* Bit 5 setzen */
```

Typkonvertierungs-Hierarchie:

`char` → `short` → `int` → `long` → `float` → `double`

Gefahrenquellen:

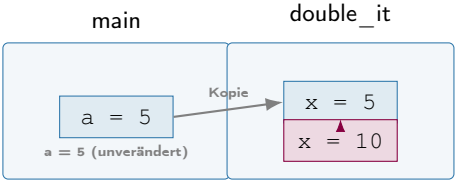
```
/* Vorzeichenwechsel */
int    neg  = -1;
uint32_t u = (uint32_t)neg; /* 2^32-1 */
/* Pointer-Aliasing (UB!) */
float  f  = 1.0f;
int    *ip = (int*)&f;  /* gefährlich */
```

Call by Value und Speicherverwaltung

Call by Value – und wie wir es umgehen

```c
/* C übergibt IMMER Kopien (Call by
    Value) */
void double_it(int x) {
    x = x * 2; /* nur lokale Kopie! */
}
int main(void) {
    int a = 5;
    double_it(a);
    printf("%d\n", a); /* Immer noch 5
     */
    return 0;
}
```

```c
/* Lösung: Pointer übergeben
   ("Call by Reference" simuliert) */
void double_it_ref(int *x) {
    *x = *x * 2;
}
double_it_ref(&a);
printf("%d\n", a);    /* 10 */
/* Structs effizient übergeben */
void sensor_process(const Sensor_t *s)
```



Arrays – immer als Pointer

```c
/* arr zerfällt zu int* */
void zero(int *arr, size_t n){
    for (size_t i=0;i<n;i++)
        arr[i] = 0;
}
int buf[8];
zero(buf, 8);  /* keine Kopie! */
```

```c
#include <stdlib.h>
/* Heap-Allokation (uninitialisiert) */
int *arr = (int*)malloc(10 * sizeof(int));
if (arr == NULL) {
    /* Fehlerbehandlung! */
    return -1;
}
/* Calloc -- mit 0 initialisiert */
int *arr2 = (int*)calloc(10, sizeof(int));
/* Realloc -- Größe ändern */
arr = (int*)realloc(arr, 20 * sizeof(int));
if (arr == NULL) { /* ... */ }
/* Freigeben */
free(arr);
arr = NULL;   /* Dangling Pointer vermeiden */
free(arr2);
arr2 = NULL;
```

Embedded-Empfehlung

In sicherheitskritischen und Echtzeitsystemen `malloc` nach Möglichkeit *vermeiden* – statische oder Stack-Allokation bevorzugen (MISRA-C Regel).

Objektorientierung in C simulieren

Structs als Klassen – das **self**-Muster

```c
#include <math.h>
/* "Klasse" = Struct als Datenkapsel */
typedef struct {
    float x, y;
} Point_t;
/* "Methoden" = Funktionen mit self-Ptr
   (analog zu C++ this-Pointer)        */
void Point_init(Point_t *self,
                float x, float y) {
    self->x = x;
    self->y = y;
}
float Point_dist(const Point_t *self,
                 const Point_t *other) {
    float dx = self->x - other->x;
    float dy = self->y - other->y;
    return sqrtf(dx*dx + dy*dy);
}
/* Verwendung */
Point_t p1, p2;
Point_init(&p1, 0.0f, 0.0f);
Point_init(&p2, 3.0f, 4.0f);
float d = Point_dist(&p1, &p2); /* 5.0
```

Namenskonventionen:

- Typ: `TypeName_t`
- Konstruktor: `TypeName_init()` bzw. `TypeName_create()`
- Methoden: `TypeName_verb()`
- Erster Parameter immer `self`
- `const` für lesende Methoden

Kapselung durch Header-Trennung

point.h – öffentliche API:
`typedef` + Prototypen

point.c – Implementierung:
Struct-Felder + Funktionskörper

⇒ Nutzer sieht *nur* das Interface

Analogie C++ ↔ C

Polymorphismus mit Funktionspointern (vtable)

```c
/* Vorwärtsdeklaration */
typedef struct Shape_s Shape_t;
/* Virtuelle Methodentabelle */
typedef float (*AreaFn_t)(const Shape_t *);
typedef void  (*DrawFn_t)(const Shape_t *);
typedef struct {
    AreaFn_t  area;
    DrawFn_t  draw;
} ShapeVTable_t;
/* Basisklasse */
struct Shape_s {
    const ShapeVTable_t *vtable;
    float x, y;
};
/* Abgeleitete "Klasse" */
typedef struct {
    Shape_t base;   /* Vererbung: zuerst! */
    float   radius;
} Circle_t;
float Circle_area(const Shape_t *self) {
    const Circle_t *c = (const Circle_t*)self;
```

```c
static const ShapeVTable_t
  circle_vtable = {
    .area = Circle_area,
    .draw = Circle_draw
};
void Circle_init(Circle_t *c,
    float x, float y, float r) {
    c->base.vtable = &circle_vtable;
    c->base.x = x;
    c->base.y = y;
    c->radius = r;
}
/* Polymorpher Aufruf */
Circle_t circ;
Circle_init(&circ, 0,0, 5.0f);

Shape_t *s = (Shape_t*)&circ;
s->vtable->area(s);  /* dispatch! */
s->vtable->draw(s);
```

Kapselung mit Opaque Pointer

Öffentlicher Header **sensor.h**:

```c
#ifndef SENSOR_H
#define SENSOR_H
#include <stdint.h>

/* Opaque Pointer -- Nutzer kennt nur den Typ, nicht die Felder! */
typedef struct Sensor_s Sensor_t;

/* Konstruktor und Destruktor */
Sensor_t *Sensor_create(uint8_t id);
void      Sensor_destroy(Sensor_t *self);

/* Öffentliche Methoden */
float Sensor_read(Sensor_t *self);
void  Sensor_calibrate(Sensor_t *self,
                       float offset);
float Sensor_last(const Sensor_t *self);

#endif /* SENSOR_H */
```

Implementierung **sensor.c**:

```c
#include "sensor.h"
#include <stdlib.h>

/* Struct-Definition: nur HIER sichtbar */
struct Sensor_s {
    uint8_t  id;
    float    offset;
    float    last_value;
};

Sensor_t *Sensor_create(uint8_t id) {
    Sensor_t *s = malloc(sizeof(*s));
    if (s != NULL) {
        s->id         = id;
        s->offset     = 0.0f;
        s->last_value = 0.0f;
    }
    return s;
}
```

Zusammenfassung

Zusammenfassung

Geschichte & Standards:

- C seit 1972 (Ritchie, Bell Labs, Unix)
- Standards: C89 → C99 → C11 → C17 → C23
- Ideal für Embedded: Hardwarenähe, Effizienz, Deterministismus

Grundkonstrukte:

- Portable Typen (`stdint.h`) bevorzugen
- `struct`, `enum`, `typedef` für Abstraktion
- `volatile` für Hardware-Register unverzichtbar
- `static` für Datenkapselung auf Dateiebene

Zeiger und Typsystem:

- `&` / `*`: Adresse und Dereferenzierung
- Pointer-Arithmetik skaliert mit `sizeof`
- Funktionspointer für Callbacks und Dispatch-Tabellen
- Casts immer explizit; `void*` für Generizität
- `const` so früh und so oft wie möglich

OOP-Muster in C:

- `struct` als Klasse, `self`-Pointer als `this`
- Funktionspointer im Struct → vtable → Polymorphismus
- Opaque Pointer → echte Kapselung
- Call by Reference: Pointer übergeben

Fragen?